A Measurement Theory of Processing

The Op as the Bit of Processing

Registry: [HOWL-MATH-15-2026]

Series Path: [HOWL-INFO-11-2026] → [HOWL-INFO-12-2026] → [HOWL-INFO-13-2026] → [HOWL-MATH-14-2026] → [HOWL-MATH-15-2026]

Date: June 2026

DOI: 10.5281/zenodo.20622727

Domain: Information Theory / Mathematics / Measurement Theory / Systems Architecture

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

---

1. The Missing Unit

In 1948, Shannon did not merely define entropy. He gave it a unit — the bit. The bit is what made information theory engineerable. Entropy as a concept describes uncertainty. Entropy measured in bits becomes a quantity you can compute, compare, optimize, and build systems against. Every communication system built since 1948 — every modem, every codec, every compression algorithm, every error-correcting code — works because the bit exists as a countable, fungible, domain-independent unit.

The prior paper in this series ([HOWL-MATH-14-2026]) defined processing entropy Hp — the work required for a processor to reduce an element to an actionable One for a given goal in a given context. It established the state space, the reduction chain, dissolution, cascade, and the bridge to Shannon's framework. It deliberately left Hp without a concrete unit. The formalism was complete. The measurement apparatus was not.

This paper provides the unit: the op.

The op is to processing what the bit is to transmission. Shannon counted bits. This paper counts ops. With a countable unit, processing entropy becomes a measurable quantity — not a theoretical construct but a number you can observe, record, compare across processors, track over time, and engineer against. The op completes the measurement apparatus that the prior paper's formalism requires.

---

2. The Op

An op is one irreducible transformation by one processor. One action. One step in a reduction chain. The smallest unit of processing that the framework needs to count.

A surgeon's scalpel makes one incision. One op. A driver's eyes move to the mirror. One op. A programmer reads one error message. One op. A mathematician evaluates one sub-expression. One op. A CPU executes one instruction. One op. A manager makes one decision. One op. A pilot classifies one radar contact. One op.

The op is abstractly fungible the way a bit is abstractly fungible. A bit transmitted over fiber optic in a nanosecond and a bit transmitted by postal mail in three days are the same bit. Shannon does not care about the medium. Shannon does not care about the duration. The bit is the unit. The medium and duration are separate concerns — they determine channel capacity and latency, not the quantity of information.

The same holds for ops. A mirror-glance op takes a fraction of a second. A diagnostic-reasoning op takes minutes. A strategic-planning op takes days. Each is one op — one irreducible transformation consuming the pipeline for its duration. The duration determines how many ops fit in the time budget, not the nature of the op itself. Duration is to ops what propagation delay is to bits — a property of the medium, not the unit.

This fungibility is what makes cross-domain comparison possible. You cannot compare a surgical incision to a database query in terms of physical action — they share nothing at the substrate level. But you can compare them as ops — each is one step in a reduction chain, each consumes the pipeline, each counts toward the processing load, each has a cost against the time budget. The framework operates at the level where the comparison is meaningful — the structural level where all processing shares the same constraint of one operation at a time.

The op count for a given element handled by a given processor IS that processor's Hp for that element. Hp is not an abstract quantity requiring a special formula to derive. It is the count of ops in the reduction chain. If a physician executes fourteen ops to reach a diagnosis, Hp = 14 for that physician, that patient, that diagnostic goal, in that clinical context. If an experienced physician has dissolved the diagnostic chain for that presentation and reaches the same diagnosis without conscious steps, Hp = 0. The number is directly observable. You count the ops.

---

3. The Time Budget

Every processor operates within a time budget. The budget is a fixed constraint determined by the domain's physics, not by the processor's preference or effort.

The highway does not slow down because the driver is texting. The opponent's aircraft does not wait because the pilot is still orienting. The patient's hemorrhage does not pause because the surgeon is deciding. The TCP connection does not extend its timeout because the server is under load. The DMA ring buffer does not grow because the consumer fell behind. The clock cycle does not stretch because the instruction is complex.

In the framework established by [HOWL-MATH-14-2026], the time budget is a Zero-external element — permanently outside the processor's operational domain. The processor cannot change it. The processor can only manage what it does within it. The time

budget is the wall. The op count is the only variable the processor controls.

This produces the fundamental inequality of processing:

total ops × average op duration ≤ time available

When the left side exceeds the right, the processor fails. The nature of the failure depends on the domain — the car drifts into the adjacent lane, the server returns a timeout, the surgeon loses the patient, the pilot loses the engagement, the ring buffer overwrites unread data. But the mechanism is identical in every case. The ops required to maintain correct processing exceeded the time available to execute them.

Every failure mode described in the prior paper reduces to this single inequality.

An immature processor fails because it has too many ops per unit. Nothing is dissolved. Every element requires a long conscious chain. The budget fills before the work completes.

A cascading processor fails because ops that were zero suddenly are not. The budget did not change. The op count spiked. The triggering event did not shrink the budget. It expanded the ops.

An over-reducing processor fails because it spent ops past the point of actionability. The budget was consumed on unnecessary reduction. The ops that would have gone to the next element never execute.

A processor wasting effort on a boundary element fails because ops are being spent on something that cannot move. The budget is consumed. Nothing changes. The ops that were needed on manageable elements never get their time slice.

And every intervention the prior paper prescribes is a strategy for keeping the left side of the inequality smaller than the right. Dissolution shrinks ops toward zero for routine elements. Correct boundary classification stops the processor from wasting ops on immovable things. Optimal reduction stops at R* instead of overshooting. Wider validity envelopes on dissolutions prevent cascade spikes. Each intervention reduces the left side. None can increase the right. The time budget is the wall.

Shannon's parallel inequality: data rate ≤ channel capacity. Same structure. Different resource. His framework tells engineers how to stay under C — the channel's bit budget. This framework tells processors how to stay under N — the time budget measured in ops. Both inequalities are hard constraints with catastrophic consequences for violation. Both are addressed by the same strategy: minimize the demand on a fixed supply.

---

4. Dissolution as Op Elimination

The prior paper defined dissolution as the collapse of a reduction chain into structure — an element's processing moving from One to Zero-absent, the pipeline freed. In op-counting terms, dissolution is the measurable reduction of op count toward zero for a specific element over time.

Consider a new driver checking the rear-view mirror. The first week, the mirror check is a conscious sequence: decide to check mirror (one op), move eyes to mirror (one op), read mirror content (one op), assess whether any vehicle poses a threat (one op), decide no action needed (one op), return eyes to road (one op). Six ops for one mirror check. Done consciously, each step consuming the pipeline.

The driver checks mirrors perhaps ten times per minute during active driving. That is sixty ops per minute spent on mirrors alone — a substantial fraction of the total processing budget, leaving limited capacity for navigation, speed management, and traffic assessment.

At six months, the mirror check has compressed. The driver glances and reads in one smooth motion. Perhaps two conscious ops — glance and assess. Twenty ops per minute. The pipeline has more room.

At five years, the mirror check costs zero conscious ops. The driver's eyes move to the mirror, the information is acquired, the assessment occurs, and the pipeline is never engaged. The check happens structurally — dissolved into the practiced pattern of eye movement and threat assessment that operates below the level of conscious attention. Zero ops. The sixty ops per minute that mirrors once consumed are now available for other elements.

This is dissolution measured. Not described qualitatively as "getting better" or "developing expertise." Measured as a count that decreases over time from six to two to zero. The dissolution curve — op count per unit plotted over repetitions or time — is the maturity trajectory made concrete. The curve starts high, decreases with practice, and approaches zero asymptotically. The shape of the curve is an empirical question. The existence of the trajectory is a structural property of processing.

R* for a mirror check is the minimum ops any competent driver needs to reliably acquire the information. Probably one — a single glance that acquires the image and extracts the threat assessment in a single operation. Below one (no check at all), the information is not acquired and the driver is operating blind. Above one (multiple conscious assessment steps per check), the driver is spending more ops than the task requires. The gap between a driver's current op count and R* is measurable inefficiency. The gap between R* and zero is what dissolution can absorb — the distance from minimum-competent to fully structural.

The dissolution rate — the speed at which op count decreases over time — varies by element, by processor, and by conditions. Some chains dissolve quickly: a new keyboard shortcut might go from three ops (remember shortcut, find keys, press) to zero ops in a few days of use. Some chains dissolve slowly: surgical knot-tying might take months of practice before the op count reaches its minimum. Some chains resist dissolution entirely: a task that changes every time it is encountered cannot form the consistent pattern that dissolution requires. The rate is domain-specific. The trajectory is universal.

Every domain has dissolution infrastructure — tools and structures built specifically to reduce op counts for processors that have not yet dissolved the chain naturally. A checklist reduces a physician's diagnostic ops by providing the reduction pipeline externally. A jig reduces an assembler's orientation ops by holding the part correctly. An IDE's au-

tocomplete reduces a programmer's typing ops by predicting the next token. A recipe reduces a cook's planning ops by pre-specifying the sequence. Each is a structural device that lowers Hp for processors whose natural dissolution has not yet reached zero. The infrastructure does not replace dissolution. It provides a bridge — a lower op count during the period before the chain dissolves naturally.

---

5. The Cascade Cost Equation

The prior paper defined cascade severity as the count of Zero-absent elements that promote to One when a context change invalidates their dissolution conditions. In op-counting terms, the cascade is a computable spike in total processing load.

The true cost of any op is not the op itself. It is:

True cost = direct ops + Σ(recovery ops per promoted element) + recruited system ops

Each term is countable. The sum is the actual budget consumption. The naive cost — one op — understates the true cost by the cascade margin.

Consider a driver on a highway at steady state. Driving skills are at Zero-absent: lane keeping, speed management, steering corrections, mirror checks, following distance. The driver is talking on a hands-free phone, also at Zero-absent — conversation flows without conscious effort. The driver is chewing gum. Also at Zero-absent. Total conscious op count: zero. Pipeline: empty. The processor has no workload.

A text message notification appears on the phone mounted on the dashboard. The driver looks at it. One op — a voluntary eye movement to read the notification.

But that one op has a cascade cost. The eye movement takes eyes off the road. Lane keeping was dissolved under the assumption that eyes are on the road. That assumption is now violated. Lane keeping promotes from Zero-absent to One. Speed management was dissolved under the assumption of steady foot pressure. The slight postural shift from looking at the phone changes foot pressure. Speed management promotes from Zero-absent to One. Following distance assessment requires forward visual field. Forward visual field is gone. Following distance promotes to One.

The conversation was dissolved under the assumption of available cognitive pipeline. The pipeline is now occupied by the text and the cascaded driving elements. The conversation drops — the driver goes silent mid-sentence. Not a decision. A mechanical consequence of pipeline overload shedding the lowest-priority stream.

Immediate state after the one-op glance: driving at One (lane keeping, speed management, following distance — three elements), reading at One, conversation dropped. Total load: four elements demanding the pipeline. Pipeline capacity: one.

The car drifts. This is the lane keeping element not receiving its time slice — the pipeline is servicing the reading op while lane keeping waits. The drift takes perhaps one to two seconds to become dangerous. This is the time budget asserting itself — at highway

speed, one second is roughly thirty meters of travel. The budget for lane deviation before crossing the lane boundary is perhaps half a second to one second depending on lane position.

The driver notices the drift. Alarm. The text is dropped — eyes return to road. Now recovery begins. But recovery from a disrupted driving state is not one op. It is a chain: reacquire lane position (one op), check speed (one op), check mirrors to assess what happened during the gap (one op), correct steering to re-center in lane (one op), scan for new threats that may have emerged during the gap (one op), re-establish following distance (one op), stabilize (one op). Seven recovery ops for driving alone.

The conversation is also disrupted. The other party is confused by the sudden silence. Re-establishing conversational context will require additional ops when the driver resumes — what were we talking about, where were we in the point being made. Perhaps three to five future ops.

Total cost of the one-op glance at the text message:

One direct op (look at phone). Three cascade promotions (lane keeping, speed management, following distance). Seven recovery ops for driving. Three to five deferred recovery ops for conversation. Approximately two seconds of degraded or zero vehicle control covering sixty meters of highway.

Total: twelve to fourteen ops for what appeared to be one op. Plus sixty meters of uncontrolled travel.

Now consider the identical action on a sofa. The driver — now a person on a sofa — picks up the phone and reads the text. One op. There are no concurrent dissolved activities that depend on visual attention. No cascade. No recovery. No budget crisis. Total cost: one op.

The op did not change. The context changed the cascade cost from zero to potentially fatal. This is the framework's explanation for why texting while driving kills people, stated not as a vague warning about "distraction" but as a computable budget violation. The one-op glance triggers a cascade whose total op cost exceeds the time budget for maintaining vehicle control at highway speed. The excess is measurable in ops, convertible to seconds, and convertible to meters of uncontrolled travel. The number can be computed in advance if you know the dissolution inventory, the validity conditions, and the recovery cost per element.

The cascade cost equation generalizes to any domain:

For any event e, in any context c, affecting any processor p:

Cascade cost(e, p, c) = Σ recovery(x_i) for all x_i where S(x_i , p, g, c) was 0a and S(x_i , p, g, c') is 1

Where recovery(x_i) is the op count required to restore element x_i to correct operation — either re-dissolving it to Zero-absent or completing its processing at One and releasing it. The sum is the total op spike. Compare the spike against the time budget. If the spike exceeds the budget, predict the failure mode from the domain's physics.

6. Op Ordering and System Recruitment

Not all ops that achieve the same result cost the same. Two paths to the same actionable One can have vastly different total costs — different direct op counts, different cascade footprints, different numbers of recruited systems generating downstream ops.

A driver needs to know what is behind the car. Two paths to the same information.

Path A: move eyes to rear-view mirror. One op. The eyes move, the image is acquired. Hands remain on the wheel. Foot remains on the pedal. Head remains forward-facing. Body position is unchanged. Zero dissolution conditions are disturbed on any other element. One system recruited — the oculomotor system — and that system's op chain is minimal. Total cost: one op, zero cascade, one system.

Path B: turn head to look behind through the rear window. Multiple ops. The head rotates, recruiting the neck musculature. The vestibular system activates because head position has changed — this system was not requested but is recruited automatically by the head movement. The vestibular activation may trigger a balance correction, recruiting the postural system. The postural shift may adjust foot pressure on the pedal, changing vehicle speed. Eyes lose the forward visual field during the turn, putting lane keeping at risk. Eyes must reacquire the forward view after the turn, costing additional ops.

Same information acquired. Path A cost one op with zero cascade and one recruited system. Path B cost multiple ops with potential cascade across several driving elements and recruited at least three systems (neck musculature, vestibular, postural) that were not needed for the goal of knowing what is behind the car.

The principle is general: **every system recruited by an op is itself an op generator.** A recruited system does not add one op. It adds that system's entire processing chain. And the processor does not control the recruited system's chain — the vestibular system does what it does once the head moves. The recruited ops are a consequence of the path chosen, not a choice the processor makes.

Minimum systems recruited is therefore a proxy for minimum total cost. Fewer systems means fewer generated ops, less blocking from systems with their own timing, less cascade surface from systems that may disturb other dissolved elements, and less unpredictable downstream cost from systems whose op chains the processor does not control.

This applies identically across every domain.

In computation: a NUMA-local memory access involves one system — the CPU core and its local memory controller. One op, fast, predictable. A remote NUMA access recruits the interconnect fabric, the remote memory controller, and the cache coherency protocol across nodes. Same data retrieved. Three additional systems recruited, each with its own latency and contention profile. The isolated op — read this memory address — is the same. The in situ cost differs by an order of magnitude based on which systems are recruited.

A blocking IO call recruits the IO scheduler, the wait queue, the context switch machinery, and the wakeup mechanism. The thread is suspended. The CPU does something else. When the IO completes, the thread is rescheduled — another context switch, another potential cache invalidation cascade. A non-blocking poll recruits only the IO status register. One system, one check, immediate return. The poll may check more frequently, but each check is cheap and involves one system. The block involves fewer checks but each one recruits half the operating system kernel.

In surgery: reaching the operative site through a natural tissue plane involves minimal system recruitment — the plane parts with gentle dissection, minimal bleeding, minimal disruption to surrounding structures. Cutting across tissue planes recruits the hemostasis system (cautery, pressure, clamps), the retraction system (assistants holding tissue aside), the irrigation system (clearing the field), and the visualization system (repositioning lights or cameras). Each recruited system has its own op chain and its own timing. The surgeon who finds the natural plane reaches the same site with fewer total ops because fewer systems were recruited along the path.

In conversation: making a point directly involves one system at the listener — their comprehension of the statement as given. Making the same point through an elaborate analogy recruits the listener's processing of the analogy itself, the mapping between the analogy domain and the target domain, potential confusion if the mapping is imperfect, and clarification requests that generate additional round-trip ops. Same point communicated. More systems recruited. Higher total cost.

The optimal op in any domain has two properties simultaneously: minimum direct ops in the chain, and minimum systems recruited per op. The best processors — the most efficient drivers, surgeons, programmers, communicators — intuitively select paths that minimize both. They reach for the mirror instead of turning their head. They find the tissue plane instead of cutting through. They use the local memory instead of the remote. They state the point instead of constructing the analogy. That selection is itself a dissolved skill — the expert does not consciously evaluate cascade footprints and system recruitment counts. The selection has dissolved to Zero-absent. The expert just picks the cheaper path because that is what expertise means, expressed in ops.

---

7. Two Counting Regimes

Not all op counting is the same. Two fundamentally different regimes exist, and confusing them produces incorrect measurements and wrong predictions.

Isolated counting measures a single task on a single pipeline with no concurrency. The task has a fixed topology. Every op can be enumerated before execution begins. The time budget is determined by the task alone. No external stream competes for the pipeline. No cascade is possible because there are no concurrent dissolved activities to disturb.

A knitter making a sweater is an isolated process. The stitch count is deterministic. The yarn changes are pre-specified. The row count is known. The entire op sequence can be written out before the first stitch is made. Nothing else is running on the knitter's pipeline

that the sweater could disturb. Nobody dies if a stitch takes an extra second. The isolated op count is the op count — there is no gap between intrinsic cost and actual cost because there is no environment to impose additional cost.

An algorithm analyzed on paper is an isolated process. A sort of N elements performs N log N comparisons and some number of swaps. A matrix multiplication performs N^3 multiply-add operations. These are deterministic, pre-computable, and environment-independent. The isolated op count is the algorithm's complexity made concrete — not an asymptotic bound but an actual count for a specific input size.

Isolated counting gives you the intrinsic cost of the task — how hard it is independent of context. This is where R* lives. The minimum correct ops to reach actionability for a given element and goal, assuming no concurrency, no contention, no cascade. R* is an isolated-regime quantity because it measures the task's irreducible cost without environmental overhead.

In situ counting measures a task executing in its actual environment — multiple concurrent streams sharing a pipeline, external events arriving on their own schedules, dissolved activities running in the background with dissolution conditions that can be violated, and shared resources with contention.

A surgeon performing an appendectomy is an in situ process. The primary surgical task is one stream. Spatial orientation — maintaining awareness of anatomical position — is another, partially dissolved. Communication with the surgical team is another, arriving when other people speak, not when the surgeon requests it. Monitoring — the anesthesia machine's beep pattern encoding oxygen saturation and heart rate — is another, dissolved to Zero-absent until the pattern changes. Delayed operations — a requested instrument that hasn't arrived yet, anesthetic that was ordered and will be delivered on someone else's timeline — are open loops, each holding a reference that consumes background awareness even at low cost.

The surgeon is not performing an appendectomy. The surgeon is managing a portfolio of concurrent streams, each generating ops on its own schedule, sharing one conscious pipeline, with interdependencies and timing constraints across streams. A high-priority alarm from the monitoring stream can preempt a lower-priority communication. An unexpected anatomical finding can promote a dissolved spatial assessment back to One. A delayed instrument arrival can stall the primary surgical stream.

A CPU under real load is an in situ process. The algorithm runs, but the scheduler preempts for other processes. Context switches invalidate cache lines — data that was at Zero-absent (in L1 cache, accessible at near-zero cost) promotes back to expensive One (must be reloaded from L2, L3, or main memory at vastly higher latency). IO requests block behind other processes' IO. Memory bus bandwidth is shared with other cores. Lock contention serializes work that was designed to be parallel.

In situ counting captures everything isolated counting captures, plus five additional cost categories:

Contention cost. Ops delayed or inflated because a shared resource is occupied by another stream. The disk serving another process's request. The memory bus saturated by

other cores. The single-lane bridge occupied by oncoming traffic. Each instance adds wait time that the isolated count does not include.

Cascade cost. Ops added when context changes invalidate dissolved elements. The context switch that pollutes the cache. The bee that promotes driving skills from Zero-absent to One. The unexpected anatomy that promotes the surgeon's spatial awareness from dissolved to conscious. Each promotion adds recovery ops to the total.

Coordination cost. Ops generated purely by managing concurrency. Acquiring locks. Synchronizing threads. Signaling between streams. Scheduling attention across multiple demands. These ops produce no progress on any individual stream — they are overhead for running multiple streams on shared infrastructure.

Blocking cost. Time spent waiting for resources held by other streams. The blocked IO call. The thread waiting for a lock. The surgeon pausing because the requested instrument has not arrived. The pipeline is idle — no op is executing — but the time budget is being consumed.

Interleave cost. Ops spent deciding which stream to service when. The surgeon deciding whether to respond to the nurse's question now or finish the current suture first. The operating system's scheduler deciding which process gets the next time slice. The air traffic controller deciding which aircraft to instruct next. Interleave decisions are ops that advance no individual stream but are necessary for managing the concurrent execution.

The gap between isolated count and in situ count is the **concurrency tax** — the total ops generated purely by executing the task while other tasks are also executing on the same pipeline.

An appendectomy in isolation might cost two hundred primary surgical ops. The same appendectomy in situ — with monitoring, communication, supply management, occasional unexpected findings, and the continuous interleave management across all streams — might cost three hundred and fifty total ops. The extra one hundred and fifty are the concurrency tax.

And in an expert surgeon, even the concurrency management is dissolved. The interleave decisions — when to listen to the monitor, when to respond to the nurse, when to pause and reassess — are structural. They cost zero conscious ops. The expert's concurrency tax is lower not because the concurrent streams are absent but because the management of those streams has dissolved to Zero-absent. The novice surgeon is consciously managing the interleave, paying full op cost for every stream-switching decision on top of the ops for the streams themselves.

So dissolution operates at two levels in the in situ regime. The primary task dissolves — fewer ops per surgical step as skill develops. And the concurrency management dissolves — fewer ops for managing the portfolio of streams. The expert has dissolved both. The novice has dissolved neither. The intermediate has dissolved some primary ops but still pays full cost for concurrency management. This is why intermediate practitioners often report feeling more overwhelmed than beginners — they have dissolved enough primary skill to see the concurrent streams clearly, but have not yet dissolved the management

of those streams. Their awareness has increased (they notice more streams) while their management capacity has not yet caught up (each stream still costs conscious ops to track).

---

8. Measuring Processing Entropy

Processing entropy Hp, defined abstractly in the prior paper, is now measurable: count the ops. For each domain, the op is a domain-specific action, the counting method is observation or instrumentation, and R* is the minimum chain for competent execution. This section establishes the measurement system for representative domains, demonstrating that Hp is observable in practice across the full range of processing substrates.

Medicine — Clinical Decision Making. The op is a clinical action: a history question asked, a physical exam maneuver performed, a lab test ordered, a differential diagnosis item considered, a diagnosis selected, a treatment initiated. Each is one op.

Counting method: direct observation with timestamp, or retrospective medical record review. The medical record captures orders, procedures, and decisions. Video recording of the encounter captures the full op sequence including physical exam maneuvers and communication that the record omits.

An experienced attending physician seeing a straightforward community-acquired pneumonia: auscultate lungs, note findings, order chest radiograph, read radiograph, diagnose pneumonia, prescribe antibiotic. Six ops. A first-year resident seeing the same patient: take comprehensive history (eight to twelve ops for review of systems), perform full physical exam (fifteen to twenty ops across all organ systems), generate broad differential (five to eight ops considering multiple diagnoses), order comprehensive lab panel and imaging (three to five ops), wait for results, interpret each result individually (five to ten ops), narrow differential through explicit reasoning (three to five ops), discuss with attending (two to three ops), finalize diagnosis, select treatment. Forty to sixty ops for the same actionable One — a treatment plan for pneumonia.

R* for community-acquired pneumonia in a classic presentation: the minimum clinical actions by any competent physician to reliably reach the correct diagnosis and treatment. Probably five to eight — targeted history of respiratory symptoms, auscultation, chest imaging, interpretation, diagnosis, treatment selection. Below that, the physician is guessing without sufficient data. Above that, the physician is performing work that does not change the outcome.

Software Engineering — Bug Resolution. The op is a developer action: read error log, reproduce the bug, open a file, read a function, form a hypothesis, set a breakpoint, run the debugger, inspect a variable, trace a call stack, identify root cause, write the fix, run tests, commit. Each is one op.

Counting method: screen recording with action annotation, or IDE telemetry that logs file opens, searches, debug sessions, and test runs. Most development environments already capture this data.

A senior developer on a familiar codebase encountering a known class of bug: read error message, open the relevant file (they know which one from the error signature), see the bug, write the fix, run tests. Five ops. A junior developer on the same bug: read error message, search codebase for the error string, open three incorrect files, read documentation for the relevant module, ask a colleague for orientation, open the correct file, read surrounding code for context, form an incorrect hypothesis, test it, see it fail, form a second hypothesis, find the bug, attempt the fix, introduce a regression, discover the regression in tests, revert, fix correctly, run full test suite, run it a second time to be sure. Twenty-five to forty ops.

R* per bug class: the minimum actions required by a developer with complete codebase knowledge to reliably identify and resolve the bug. IDE tooling, linters, type systems, and automated testing are dissolution infrastructure — each reduces the developer's op count by collapsing manual steps into structural checks.

Aviation — Combat Engagement. The op is a pilot action: check radar, identify a contact, classify the threat, assess geometry, select weapon, compute engagement envelope, maneuver for position, acquire lock, fire, assess result. Each is one op.

Counting method: cockpit video recording, heads-up display tape, flight data recorder, and post-mission debrief reconstruction. Military aviation already instruments every training sortie at this level of granularity. Every action is reconstructable from the combined data sources.

A veteran fighter pilot engaging a single threat with a familiar profile: radar contact, classify (dissolved — the profile is recognized structurally), select weapon (dissolved — the correct weapon for this threat class is structural), maneuver (dissolved — the aircraft handling is structural, only the tactical vector is conscious), acquire, fire. Perhaps three to four conscious ops — the rest dissolved. A student pilot in the same scenario: detect contact (one op), read range and bearing (one op), assess closure rate (one op), recall threat classification criteria (multiple ops), classify (one op), recall weapons employment procedures (multiple ops), select weapon (one op), recall engagement geometry requirements (multiple ops), plan maneuver (multiple ops), execute maneuver while consciously managing aircraft (multiple ops — flying and tactical thinking competing for the pipeline), attempt lock (one op), verify parameters (one op), fire (one op), assess (one op). Twenty to thirty ops, many of them slowed by the competition between aircraft handling and tactical processing on a single pipeline.

R* per tactical scenario: the minimum actions to achieve the engagement objective. This is precisely what advanced weapons schools teach — the optimal reduction chain for each class of encounter, practiced until dissolved. The gap between a graduate's op count and the student's is the school's measurable output.

Computation — Algorithm Execution. The op is a CPU instruction or a meaningful grouping — a memory access, an arithmetic operation, a branch decision. Countable by hardware performance counters at cycle-level precision.

Counting method: the most precisely instrumented domain in existence. Hardware performance counters report instructions executed, cache hits and misses at every level,

branch predictions and mispredictions, context switches, and instructions per cycle. Software profilers report function-level and line-level timing. The measurement infrastructure is built into the hardware itself.

The isolated op count for an algorithm is its complexity made concrete. Not O(N log N) as an asymptotic upper bound but the actual instruction count for this specific N on this specific input. A sort of one million integers might execute exactly seventeen million comparisons and four million swaps.

But computation introduces a property not present in other domains: ops have vastly different time costs depending on data locality. A compute op — an addition, a comparison — costs one clock cycle. An L1 cache hit costs approximately four cycles. L2 costs approximately twelve. L3 costs approximately forty. Main memory costs approximately two hundred cycles. A disk access costs millions of cycles. A network round-trip costs tens of millions.

These are all nominally one op — "read this value" — but they vary by four orders of magnitude in time cost. So the raw op count alone does not determine the time budget consumption. The weighted op count — each op multiplied by its latency tier — is the true cost. An algorithm that executes more total ops but keeps its data in L1 cache may consume less time budget than an algorithm with fewer ops that repeatedly misses to main memory.

This means R* in computation has two components: the minimum op count (algorithmic complexity) and the minimum weighted cost (data locality optimization). The algorithm with optimal asymptotic complexity but poor cache behavior may be slower in practice than a suboptimal algorithm with excellent locality. Both dimensions must be optimized.

Driving — Vehicle Operation. The op is a driver action: scan a mirror, check speed, adjust steering input, apply brake, apply throttle, activate turn signal, execute lane change, check blind spot. Each is one op.

Counting method: eye tracking hardware (measures saccades and fixations), vehicle telemetry (measures steering inputs, pedal forces, speed changes), and physiological monitoring (measures cognitive load indicators). All three are standard in driving research and have been used in hundreds of published studies.

An experienced driver on a familiar commute in light traffic generates very few conscious ops per mile — most driving actions are dissolved. An unfamiliar route in heavy traffic generates constant conscious ops — every intersection is a decision, every lane change is a planned sequence, every vehicle in the vicinity is a tracked object.

R* per mile on a given road type: the minimum conscious actions required by a fully competent driver in normal conditions for that road. A straight highway segment in light traffic has a very low R* — perhaps a few steering corrections and speed adjustments per mile. A dense urban intersection has a high R* — multiple mirror checks, pedestrian scans, signal changes, and turning decisions.

Manufacturing — Assembly. The op is an assembly action: pick a part, orient it, position it, fasten it, inspect it, advance to the next station. Each is one op.

Counting method: time-and-motion study — the oldest systematic op-counting methodology, developed by Taylor and Gilbreth over a century ago. Video recording with frame-by-frame analysis. Modern variants use motion capture and automated action recognition.

An experienced assembler picks, positions, and fastens in a continuous flow — minimum ops, each dissolved to motor memory. A new worker picks, checks orientation against the assembly diagram, rotates the part, checks again, positions, adjusts, fastens, checks torque, inspects visually. Three times the ops for the same unit produced.

R* per unit: the minimum physical actions for correct assembly assuming dissolved motor skills. Jigs, fixtures, and specialized tooling are dissolution infrastructure — they eliminate ops by embedding requirements into physical structure. A jig that holds the part in the correct orientation dissolves the orient-and-verify ops to zero. A torque-limiting driver dissolves the verify-torque op to zero. Each piece of tooling is a structural element that converts an op from One to Zero-absent.

Air Traffic Control. The op is a controller action: scan the radar scope, identify a conflict pair, issue an instruction, verify readback, update the mental model, hand off an aircraft to the next sector. Countable from audio recordings (every transmission is logged and timestamped) and eye tracking on the radar display.

Cooking. The op is a kitchen action: pick up an ingredient, measure, cut, add to the vessel, adjust heat, stir, taste, season. Countable from video recording. Mise en place is dissolution infrastructure — pre-executing measurement and preparation ops so the cooking phase has a shorter chain.

Mathematics. The op is a symbolic manipulation: apply an operation, substitute, simplify, factor, evaluate. Countable from written work — every line on the page is one or a small number of ops. The gap between student and professor for the same problem is directly visible in the line count.

Customer Support. The op is a support action: read the ticket, classify the issue, search the knowledge base, attempt a solution, verify with the customer, escalate, resolve. Countable from ticketing system logs with timestamps on every action.

The common structure across all domains: the op is an irreducible action by the processor. It is countable by direct observation or existing instrumentation. The count decreases with expertise. The minimum count is R*, definable per element per goal. Dissolution drives the count toward zero for routine elements. The time budget is fixed by domain physics. Failure is the inequality violated — total ops exceeding the budget. The measurement framework is the same. Only the ops are domain-specific.

---

9. The Cascade in Ops

The cascade cost equation from Section 5, combined with the in situ counting regime from Section 7, produces a complete measurement of disruption cost in any domain. Three examples illustrate the full accounting.

Example 1: The texting driver. Fully counted in Section 5. One direct op triggers a cascade of three promotions from Zero-absent to One, producing twelve to fourteen total ops and approximately two seconds of degraded vehicle control at highway speed. The time budget for safe lane maintenance at highway speed is approximately half a second to one second. The cascade exceeds the budget by two to four times. The framework predicts a lane departure, which is exactly what occurs.

Example 2: A server under traffic spike. A web service handles requests using an auto-scaling system that has been dissolved to Zero-absent — it provisions additional capacity without human intervention when load increases. The service runs at steady state: isolated op cost per request is twenty ops (parse, authenticate, query, compute, serialize, respond), in situ cost is approximately twenty-five ops accounting for normal contention on shared database connections. Time budget per request: five hundred milliseconds (the configured timeout).

A traffic spike arrives — ten times normal load. Auto-scaling triggers. But a configuration limit — set months ago and forgotten — caps the maximum instance count below what the spike requires. Auto-scaling fails silently. It was Zero-absent. Now it has promoted to One — someone must diagnose why scaling stopped and manually override the limit.

Meanwhile, every request is competing for fixed resources. Database connection pool is exhausted. Requests queue. Each queued request's contention cost increases as the queue deepens. A request that took twenty-five ops in situ now takes twenty-five ops plus one hundred cycles of wait time for a database connection. The wait time alone exceeds the five hundred millisecond timeout. Requests begin failing.

The monitoring system fires alerts. An engineer — who was working on a feature (manageable One for a different goal) — must context-switch to the incident. That context switch has its own cascade: the feature work promotes from active One to suspended Infinity, the engineer's mental model of the incident environment must be built from scratch (multiple reduction ops to read dashboards, correlate metrics, identify the bottleneck).

Total cascade cost: one configuration limit (the triggering event) cascades through auto-scaling failure (one promotion), resource exhaustion (contention cost multiplied across all active requests), timeout failures (customer-visible), alert storm (multiple monitoring elements promoting from Zero-absent to One), engineer context switch (primary task suspended, incident response chain initiated). The cascade count — number of elements that promoted from Zero-absent to One — is perhaps five to eight. But the total op cost includes the contention multiplier across every active request, turning a five-element cascade into thousands of failed requests.

Example 3: Unexpected anatomy in surgery. A surgeon performing a laparoscopic cholecystectomy (gallbladder removal) encounters an anatomical variant — the cystic duct joins the common bile duct at an unusual angle, obscured by adhesions from prior inflammation.

The primary surgical task promotes from partially dissolved (routine cholecystectomy is well-practiced) to fully conscious. Every dissection step now requires explicit anatomical assessment. Op count per dissection step increases from one (dissect along the known

plane) to three or four (dissect, inspect, verify anatomy, confirm safe before proceeding).

Spatial orientation promotes from Zero-absent to One. The surgeon's mental model of the expected anatomy no longer matches what is visible. Rebuilding the model requires multiple ops: identify landmarks, compare to known variants, mentally rotate the anatomy to understand the variant's geometry.

Communication ops spike. The surgeon requests intraoperative cholangiography (imaging of the bile ducts) — a request that involves communicating with the radiology technician, repositioning equipment, and waiting for the imaging to be performed. Each is an op. The wait is blocking time.

Monitoring attention increases. The anesthesiologist is asked for an update on patient status because the case will run longer than planned. The scrub nurse is asked to prepare additional instruments that the standard tray does not include. Each interaction is ops on the surgeon's pipeline plus ops on the other team members' pipelines.

Total cascade: the anatomical variant (triggering event) promotes four to six elements from Zero-absent or partial dissolution to full One across surgical task, spatial orientation, communication, and monitoring. The surgeon's op count per minute might double or triple for the affected portion of the case. The time budget — patient tolerance of anesthesia, available operating room time — is fixed. If the cascaded op count pushes the total case time beyond the patient's physiological tolerance, the failure is a surgical complication.

In each example, the same accounting applies. Count the direct ops. Count the promotions. Count the recovery ops per promotion. Count the recruited system ops. Count the contention and blocking costs. Sum. Compare against the time budget. The comparison predicts whether the processor succeeds or fails, and the magnitude of the gap predicts the severity of the failure.

---

10. Throughput in Op Units

The prior paper's Theorem 1 states that system throughput is bounded by the ratio of Zero-absent elements to total elements. In op units, this becomes concrete and calculable.

A processor's total capacity is determined by its time budget divided by its average op duration:

Capacity = $N / \bar{d}$

Where N is the time available and $\bar{d}$ is the mean duration of a single op for this processor. This is a fixed number — the maximum ops the pipeline can execute in the available time. It is the processing analog of Shannon's channel capacity C — the maximum rate of reliable operation.

The demand on the pipeline is the sum of ops across all active elements:

Demand(t) = Σ ops(x_i) for all x_i where S(x_i) $\in \{\infty, 1\}$

Elements at Zero-absent contribute zero ops to demand. Elements at Zero-external contribute zero — no ops are possible. Only elements at Infinity (being reduced) and One (being operated on) consume the pipeline.

Throughput — units successfully completed per time period — equals the capacity divided by the average ops per unit:

Throughput = Capacity / $\bar{H}p$ = N / ($\bar{d} \times \bar{H}p$)

Where $\bar{H}p$ is the average processing entropy across all elements the processor handles.

As dissolution progresses, $\bar{H}p$ decreases. Elements that once cost twenty ops now cost five, then two, then zero. The same capacity services more units per period. Throughput rises — not because the processor is faster ($\bar{d}$ has not changed) but because the processor asks less of itself per unit.

The mature system's advantage is now precisely quantifiable. If a novice surgeon averages forty ops per case step and an expert averages four, the expert has ten times the throughput on the same pipeline. Not ten times faster per op — the same speed per op. Ten times fewer ops per unit. Same pipeline, same clock, same budget, ten times the output.

This is the universal statement of a principle every programmer already knows: you cannot make the CPU faster, but you can ask it to do fewer things. Performance optimization is not about increasing $\bar{d}$. It is about decreasing $\bar{H}p$. Every optimization technique — caching, loop elimination, algorithmic improvement, batch processing — is a strategy for reducing the ops demanded per unit of output.

Caching is dissolution. The reduction chain ran once. The result was stored. Now it is retrieved structurally. Hp drops to zero for cached elements.

Algorithmic improvement is chain replacement. Bubble sort and quicksort reach the same actionable One — a sorted array. Bubble sort costs N^2 ops. Quicksort costs N log N ops. Same result. Fewer ops. Same capacity now fits more sorts.

Loop elimination is chain shortening. A redundant computation inside a loop costs M ops per iteration. Moving it outside costs M ops once. The per-iteration cost drops. Total ops decrease. Throughput increases.

Branch prediction in CPUs is speculative dissolution. The processor bets that a particular reduction path will be needed and pre-executes it. If the bet is correct, the ops are already complete when the pipeline reaches that point — Hp for the predicted path is zero. If the bet is wrong, the speculated work is discarded and the correct path must execute at full op cost. The misprediction is a small cascade — dissolved prediction promotes back to One.

In every domain, the throughput equation is the same: do fewer ops per unit, get more units done. The framework gives this principle a universal mathematical statement and a measurement methodology: count the ops, track the dissolution, compute the throughput, compare across processors and across time.

---

11. Communication Cost in Ops

The prior paper's Theorem 4 states that total communication cost between two processors equals the sender's processing cost plus Shannon's channel cost plus the receiver's processing cost. In op units, this becomes a concrete cross-unit equation:

Cost(A → B) = ops(A, encoding) + bits(channel) + ops(B, decoding)

Two different countable units compose the total cost — ops at the endpoints, bits in the channel. Shannon gave the tools to minimize the middle term. This framework gives the tools to minimize the outer terms. The total cost spans both frameworks.

Consider four scenarios between communicating processors who share a technical vocabulary — say, two engineers discussing a system architecture.

Expert to expert. The sender has dissolved the vocabulary. Encoding cost: near zero ops — the technical terms are at Zero-absent, the message composes structurally without conscious word-by-word construction. Channel cost: the bits to transmit the message — Shannon's domain, fixed by the message length and the channel characteristics. Receiver decoding cost: near zero ops — the receiver has the same dissolved vocabulary, the message decompresses structurally. Total cost is dominated by channel cost. This is the scenario where Shannon's framework alone is sufficient to analyze the communication.

Expert to novice. Same sender encoding cost: near zero ops. Same channel cost: same bits. But the receiver's decoding cost is high. Every technical term is at One for the novice — each must be consciously processed, meaning retrieved or inferred from context, associations built. A five-word technical sentence that the expert receiver processes at zero ops might cost the novice thirty ops — six ops per word for unfamiliar terms. The total cost is dominated by the receiver's processing entropy. The channel is perfect — Shannon's framework says the communication is reliable. But the communication fails at the endpoint because the receiver's Hp exceeds the receiver's time budget for the conversation.

Novice to expert. The sender's encoding cost is high: the novice struggles to compose the message, searching for words, uncertain of terminology, multiple drafts. Many ops before the message even enters the channel. Channel cost: same. Receiver decoding cost: near zero — the expert processes the message structurally regardless of how much effort it cost the sender to produce it. Total cost is dominated by the sender's processing entropy.

Novice to novice. Both endpoints are expensive. High encoding ops. High decoding ops. The channel cost — the middle term that Shannon optimizes — is a small fraction of the total. Optimizing the channel (better audio quality, higher bandwidth video) would improve total cost marginally. The bottleneck is at both endpoints.

The practical implication: for most real-world communication, the processing terms dominate the channel term. Shannon solved the term that, in many human communication scenarios, contributes the least to total cost. This does not diminish Shannon's achievement — the channel solution enabled the infrastructure on which all modern communication runs. It does clarify where the remaining optimization opportunity lies: at the endpoints that Shannon explicitly excluded.

This explains phenomena that Shannon's framework alone cannot address. A meeting where all participants share a dissolved vocabulary processes efficiently — low Hp at all endpoints, total cost is channel-dominated. The same presentation delivered to a mixed audience of experts and novices is expensive — not because the channel degrades (same slides, same audio) but because the processing entropy differential across receivers is large. Some receivers are at zero ops per term. Others are at five or ten. The meeting's total communication cost is the sender's encoding ops plus the channel bits plus the sum of all receivers' decoding ops. Minimizing the sender's encoding or the channel quality addresses the smallest terms.

Documentation written by experts for experts is Shannon-optimal — compressed, terse, efficient on the channel, negligible encoding cost, negligible decoding cost for the intended audience. The same documentation presented to novices is a processing disaster — every compressed term that saved channel bits costs the novice multiple decoding ops. The documentation that is optimal for total cost to a novice audience looks inefficient by Shannon's measure — longer, more redundant, more explicit — but each added word is dissolution infrastructure for the receiver, reducing the receiver's Hp by providing the decompressed associations that the expert receiver already has at Zero-absent.

---

12. Formal Measurement Definitions

The following definitions collect the measurement apparatus established throughout this paper into a single formal reference.

Op. The irreducible unit of processing. One transformation by one processor. Countable, abstractly fungible across domains. Duration varies by substrate and domain. The op is to processing what the bit is to transmission.

Hp(x | p, g, c) = k. Processing entropy. The number of ops processor p requires to reduce element x to an actionable One for goal g in context c. When k = 0, the element is at Zero-absent — dissolved into structure. Directly measurable by counting the ops in the reduction chain.

R*(x, g) = min k such that A(r_k (x), g) = true and all r_i correct. The minimum op count for any competent processor to reach actionability for element x and goal g. The theoretical lower bound on Hp. An isolated-regime quantity — measured without concurrency or environmental overhead.

Time budget N. The maximum duration available for processing in a given domain and context. Determined by domain physics. Unmanageable — Zero-external in the prior paper's framework.

Processing capacity = N / $\bar{d}$. The maximum ops that fit in the time budget, where $\bar{d}$ is the average op duration for the processor. The processing analog of Shannon's channel capacity C.

Processing demand at time t = Σ ops(x_i) for all elements at Infinity or One. The total ops the pipeline must execute. Must remain within capacity to avoid failure.

Throughput = N / ($\bar{\mathbf{d}} \times \bar{\mathbf{H}}\mathbf{p}$). Units completed per time period. Increases as average Hp decreases through dissolution.

Cascade cost of event e = direct ops + Σ recovery($\mathbf{x}_i$) for all elements promoted from Zero-absent to One by the context change. Each recovery(x_i) is the op count to restore element x_i to correct operation. The sum is the total op spike from the event. Measurable by counting promotions and their individual recovery chains.

Concurrency tax = in situ op count − isolated op count. The additional ops generated by the execution environment — contention, cascade, coordination, blocking, and interleave costs. Measurable by comparing the same task performed in isolation versus in its real operating context.

Dissolution rate = −dHp/dt. The rate at which op count decreases for a given element through practice. Measurable as the slope of the ops-per-unit curve over time. Varies by element, processor, and conditions.

Communication cost = ops(sender, encoding) + bits(channel) + ops(receiver, decoding). The full cost of transmitting actionable information from one processor to another. Spans both Shannon's framework (bits) and this framework (ops). Measurable by counting encoding ops, computing channel cost per Shannon, and counting decoding ops.

Failure condition: Σ ops $\times$ $\bar{\mathbf{d}}$ > N. Total op cost exceeds time budget. The universal failure inequality. Every failure mode in the framework — immaturity, cascade, over-reduction, misclassified boundary — reduces to a specific mechanism by which the left side exceeds the right.

---

13. Scope and Exclusions

This paper defines measurement. It does not define what the ops mean in any specific domain. The op is domain-specific the same way the bit is medium-specific. A bit over copper and a bit over radio are counted the same way. An op in surgery and an op in programming are counted the same way. The counting framework is universal. The ops are local.

This paper does not define optimal strategies for any domain. It defines how to measure the cost of whatever strategy a processor employs, and how to compare that cost against the theoretical minimum R* and against the time budget N. Whether a surgeon should use a particular approach to reach the operative site is a surgical question. How many ops the approach costs and whether those ops fit in the time budget is a measurement question. This paper addresses the latter.

This paper does not prove that ops are the only relevant unit. Op count determines whether the time budget is met. Op duration determines how much budget each op consumes. Op weight — the latency tier in computation, the complexity variation across domains — determines the true time cost of a counted op. The relationship between count, duration, and weight is domain-specific. This paper establishes count as the primary unit and

notes that weighting may be necessary in domains where op durations vary by orders of magnitude.

Several directions remain open for subsequent work.

Op weighting formalization. In computation, ops have latency tiers (L1 through main memory through disk) spanning four orders of magnitude. Other domains may have analogous tiers — a surgical op that encounters unexpected bleeding costs more time than one that proceeds cleanly, though each is nominally one op. A universal weighting scheme may exist, or weighting may be irreducibly domain-specific. The question is empirical.

Dissolution rate modeling. The dissolution curve — ops per unit decreasing over repetitions — has a characteristic shape. Is the shape universal across domains? Exponential decay, power law, or something else? Does the shape depend on the complexity of the chain being dissolved, the consistency of the context during practice, or inherent properties of the processor? Formalizing the dissolution rate would enable prediction of when a chain will reach Zero-absent, the same way channel coding theory enables prediction of the minimum encoding length.

Cascade prediction from structural analysis. The cascade cost is computable after the fact — count the promotions and their recovery costs. Can it be computed before the fact — predicting cascade severity for events that have not yet occurred, based on the dissolution inventory and validity conditions? This would be the processing equivalent of reliability engineering — predicting failure modes from structural analysis rather than waiting for failures to occur.

R* derivation for specific domains. R* is defined as the minimum correct op chain. For some domains — mathematics, certain well-defined computational problems — R* is provably derivable. For others — medicine, combat aviation — R* may only be estimable from large-sample observation of expert performance. The question of which domains admit provable R* and which admit only empirical R* is open.

Network processing entropy. Multiple processors communicating and collaborating, each with their own dissolution states and cascade profiles. The processing entropy of the network as a whole may not be the sum of individual Hp values. Cascade can propagate across processors — one processor's failure spikes the ops on connected processors. Shared dissolution infrastructure (shared codebooks, shared training, shared tools) reduces Hp across the network simultaneously. The network properties of processing entropy are not addressed in this paper and constitute a natural extension.

References

Shannon, C.E. "A Mathematical Theory of Communication." *Bell System Technical Journal*, 27(3), 379–423, July 1948.

[HOWL-MATH-14-2026] "A Mathematical Theory of Processing: Formalizing What Shannon Excluded." HOWL-MATH-14-2026. June 2026. DOI: 10.5281/zenodo.PENDING.

[HOWL-INFO-13-2026] "The Six States of Information: Every Problem Lives in One of Six Cells — Most Failures Come from Putting It in the Wrong One." HOWL-INFO-13-2026. June 2026. DOI: 10.5281/zenodo.20615401.

[HOWL-INFO-12-2026] "Information Processing Requires Reduction to Cardinality One: The Universal Bottleneck of Information Processing." HOWL-INFO-12-2026. June 2026. DOI: 10.5281/zenodo.20615400.

[HOWL-INFO-11-2026] "The Relationship of Zero, One, and Infinity in Information Processing: The Intrinsic Cardinalities of Computation." HOWL-INFO-11-2026. June 2026. DOI: 10.5281/zenodo.20615399.

[**HOWL-MATH-15-2026**] A Measurement Theory of Processing. Github: https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026

[**HOWL-INFO-11-2026**] The Relationship of Zero, One, and Infinity in Information Processing. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026

[**HOWL-INFO-12-2026**] Information Processing Requires Reduction to Cardinality One. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026

[**HOWL-INFO-13-2026**] The Six States of Information. Github: https://github.com/ghowland/papers/tree/main/papers/IN
INFO-13-2026

[**HOWL-MATH-14-2026**] A Mathematical Theory of Processing. Github: https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-14-2026